

OS Basics and Process

Youngjae Kim (Ph.D.)

Sogang University

Linux 운영체제 및 응용 (GITF315-01)

Today

- **OS Basics**
- Virtualization
- Process, Thread, and Concurrency Problem

Operating System (OS) and Virtualization

■ Responsible for

- Making it easy to **run** programs
- Allowing programs to **share** memory
- Enabling programs to **interact** with devices

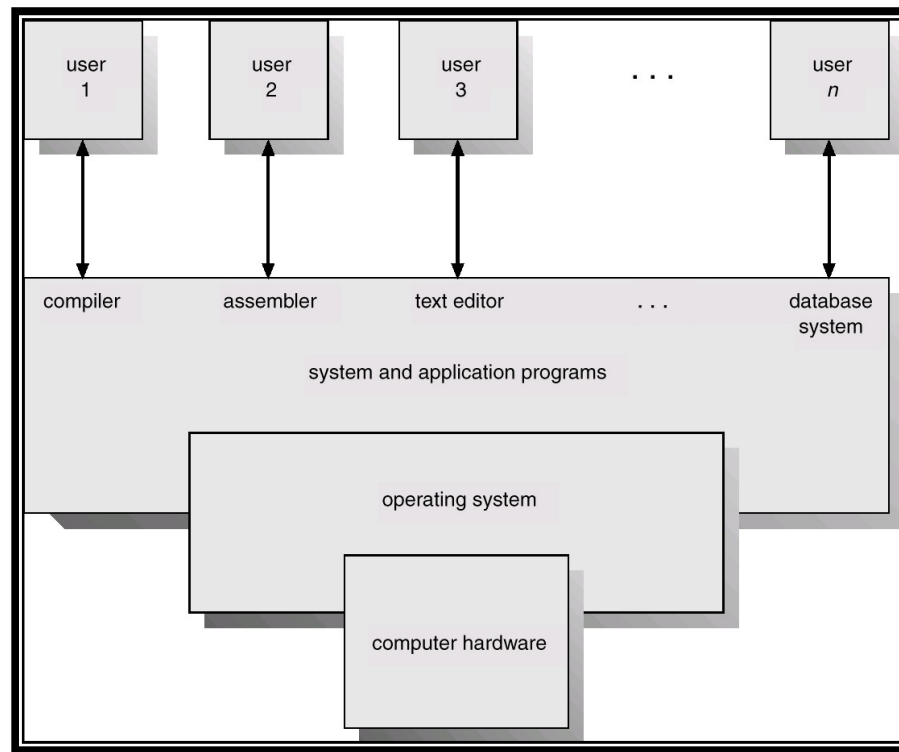
■ OS makes sure the system operates **correctly** and **efficiently**.

일관성 효율성

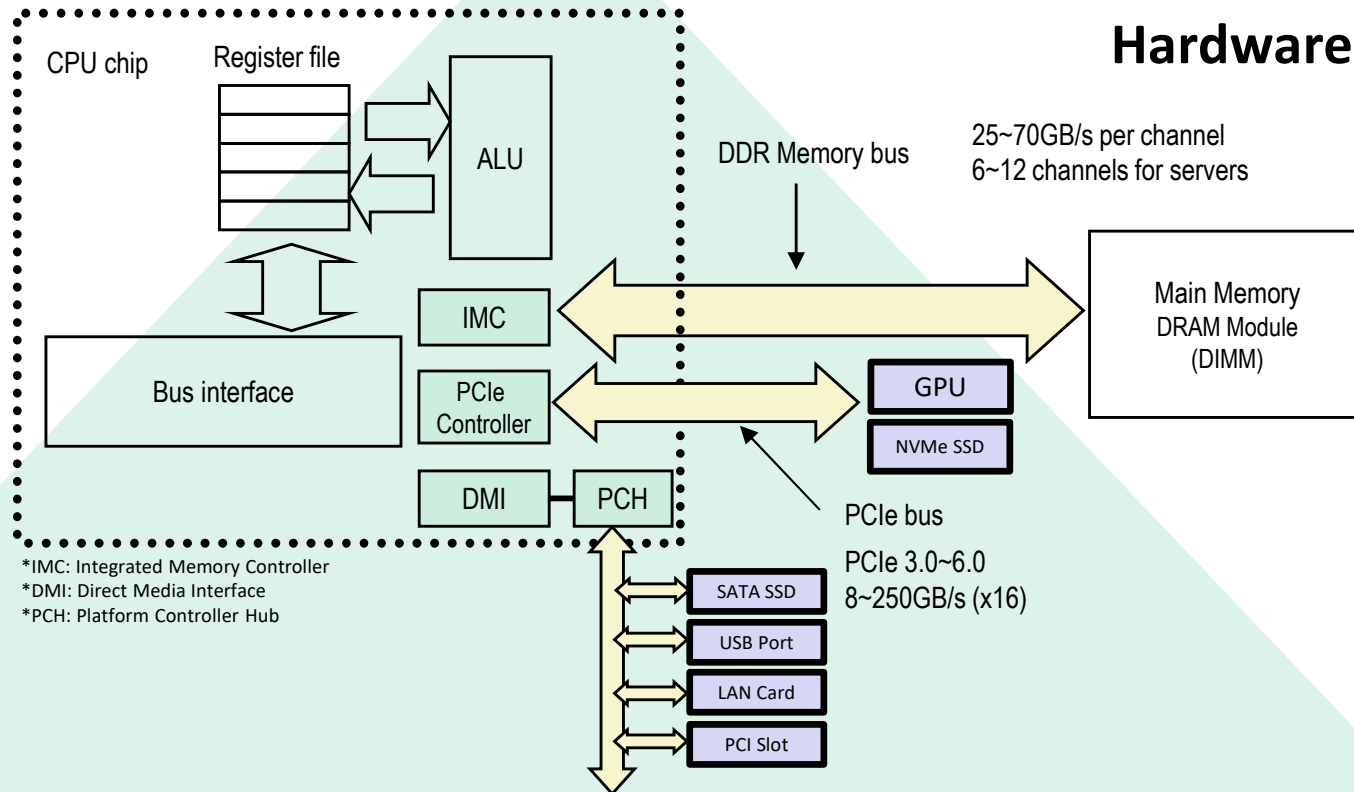
■ The OS takes **a physical resource** and transforms it into a **virtual form** of itself.

- **Physical resource:** Processor, Memory, Disk ...
- The virtual form is more general, powerful and easy-to-use.
- The OS is referred to as a **virtual machine**.

Abstract View of System Components



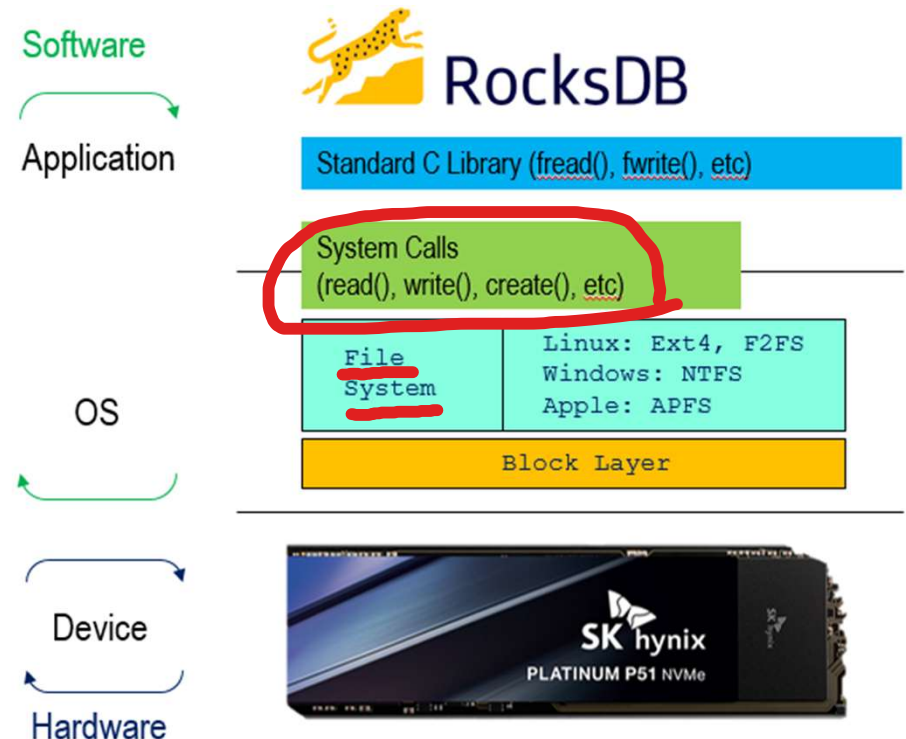
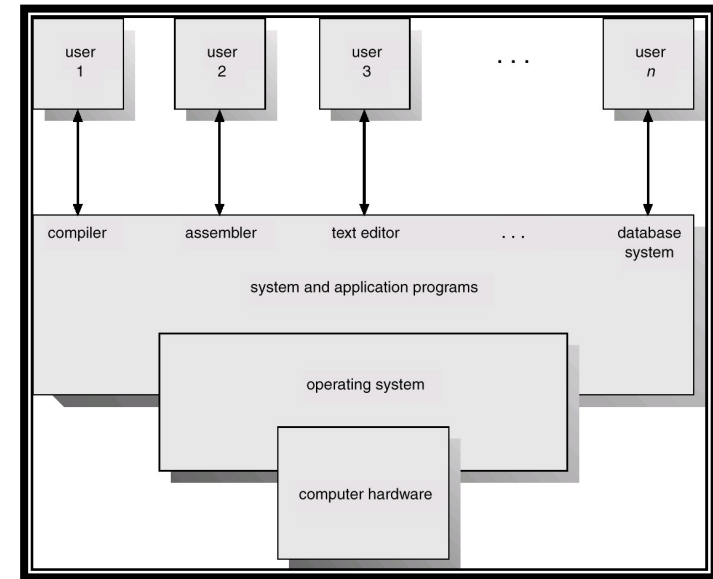
Modern Computer Hardware



System Call

메모리에 적재가 되어있으면 프로세스
안되어있으면 프로그램.

- System call allows user to tell the OS what to do.
 - System calls provide an interface between OS and a process.
 - The OS provides some interface (APIs, standard library).
 - A system call is "a program" that request a service from OS.
- What are the services of system calls?
 - Executing and creating new processes
 - Accessing hard disk



System Call (Cont.)

- A typical OS exports a few hundred system calls.
 - Process Management
 - e.g., `fork()`, `execve()`
 - File System Access
 - e.g., `open()`, `read()`, `write()`, `seek()`
 - Memory Management
 - e.g., `brk()`, `mmap()`, `mprotect()`
 - Device I/O
 - e.g., `ioctl()`, `select()`, `epoll()`
 - Communication
 - `socket()`, `bind()`, `connect`
 - Protection and Access Control
 - `chmod()`, `chown()`, `setpid()`

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the man page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

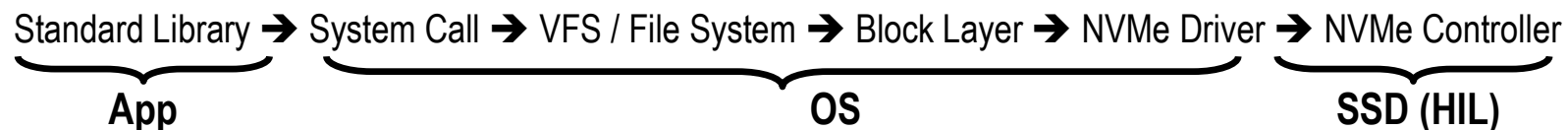
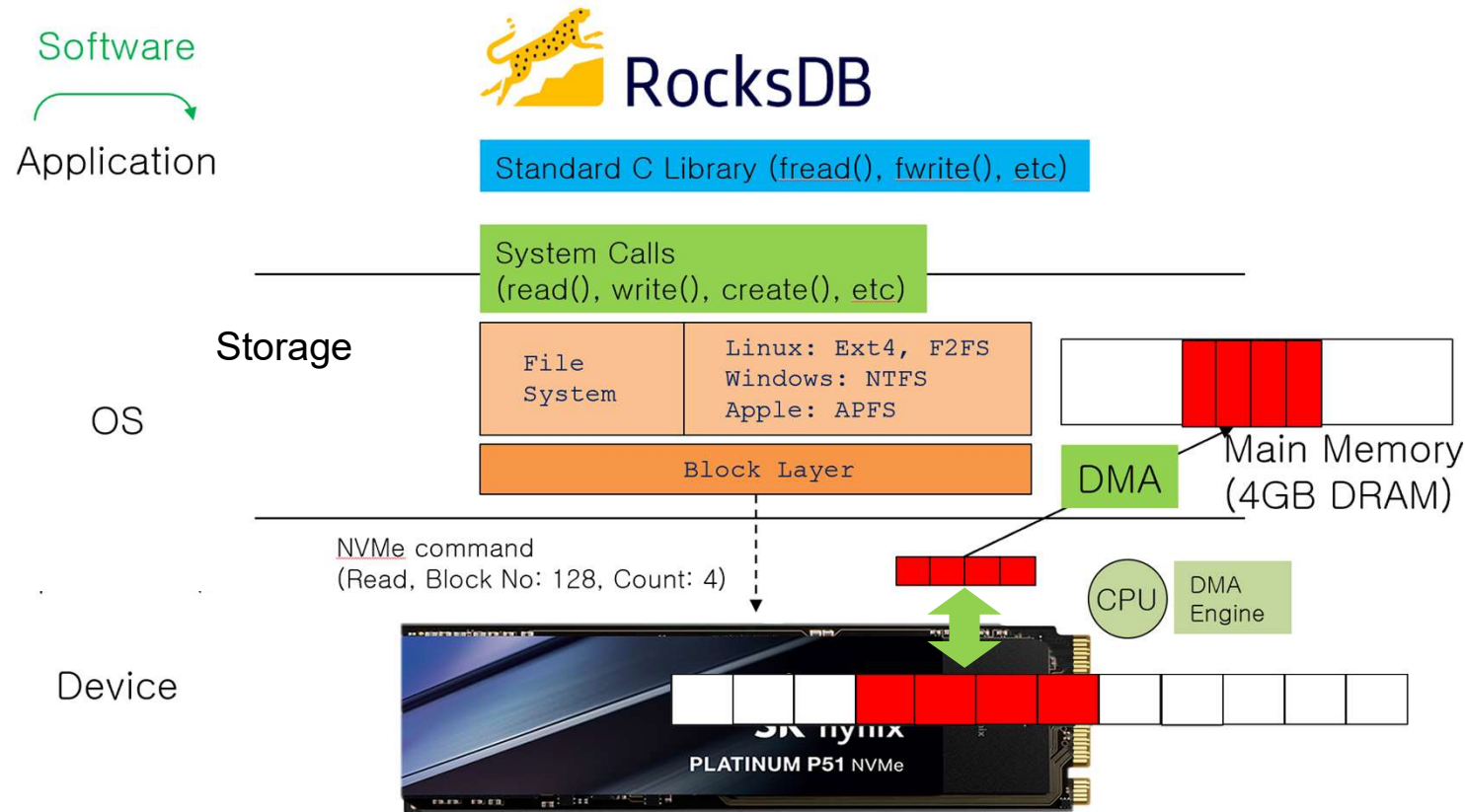
#include <unistd.h>		
ssize_t	read	(int fd, void *buf, size_t count)
return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

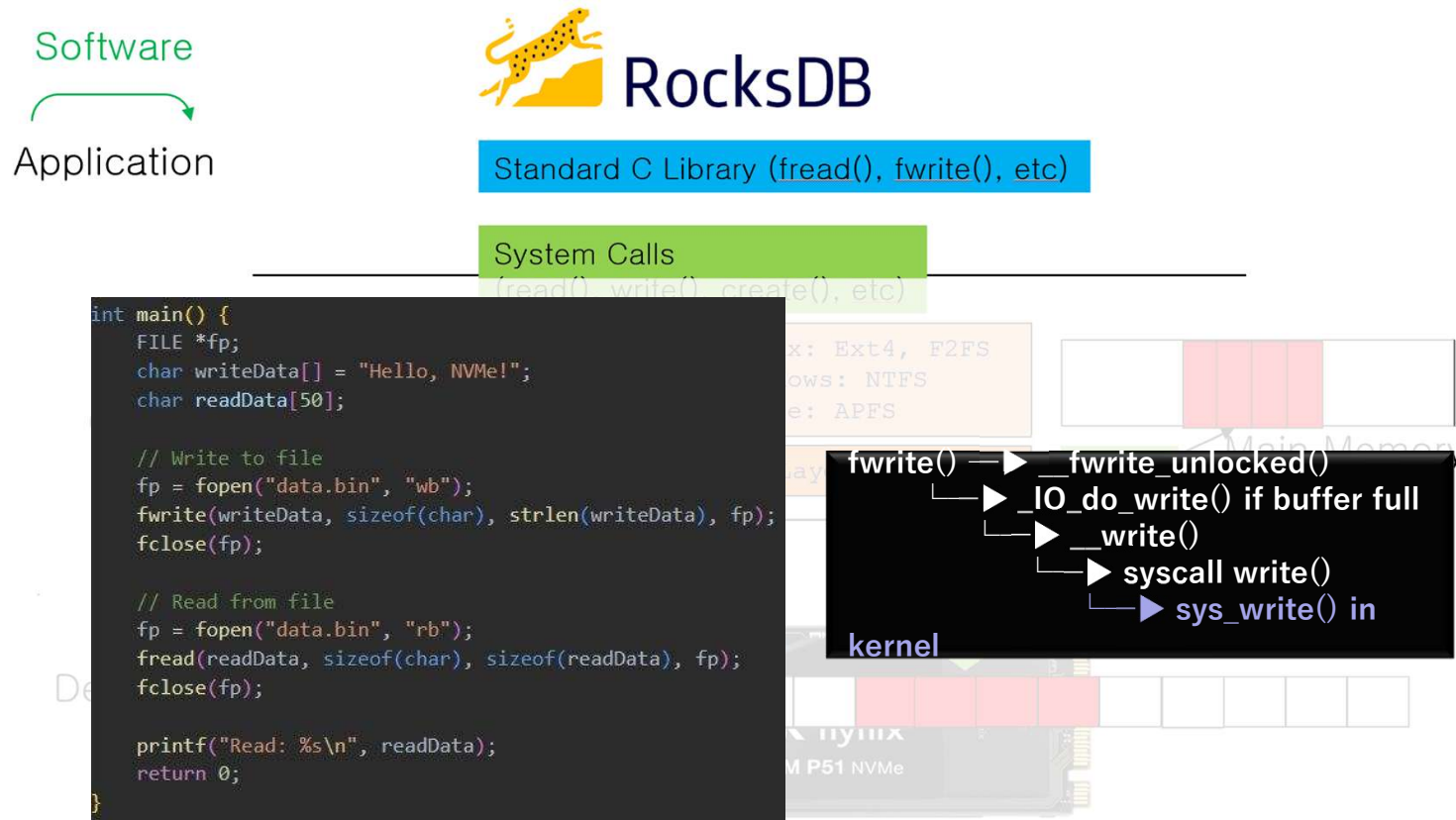
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

Example) Storage Stack for NVMe SSDs



Example) Storage Stack for NVMe SSDs



The OS is a resource manager.

- The OS manage resources such as *CPU*, *memory* and *disk*.
- The OS allows
 - Many programs to run → Sharing the CPU
 - Many programs to *concurrently* access their own instructions and data → Sharing memory
 - Many programs to access devices → Sharing disks

Virtualizing the CPU

- **The system has a very large number of virtual CPUs.**
 - Turning a single CPU into a seemingly infinite number of CPUs.
 - Allowing many programs to seemingly run at once
→ **Virtualizing the CPU**

Virtualizing the CPU (Cont.)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <sys/time.h>
4      #include <assert.h>
5      #include "common.h"
6
7      int
8      main(int argc, char *argv[])
9      {
10         if (argc != 2) {
11             fprintf(stderr, "usage: cpu <string>\n");
12             exit(1);
13         }
14         char *str = argv[1];
15         while (1) {
16             Spin(1); // Repeatedly checks the time and
17                     // returns once it has run for a second
18             printf("%s\n", str);
19         }
20         return 0;
21     }
```

Simple Example(cpu.c): Code That Loops and Prints

Virtualizing the CPU (Cont.)

■ Execution result 1.

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
^C
prompt>
```

Run forever; Only by pressing "Control-c" can we halt the program

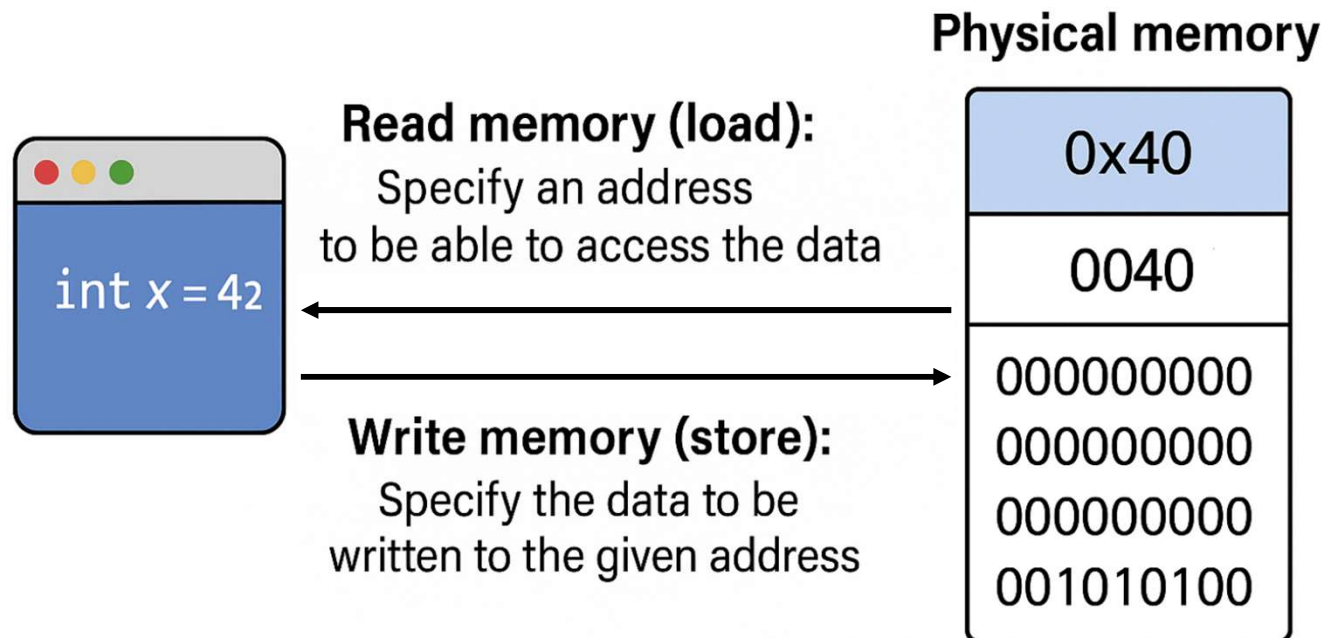
Virtualizing the CPU (Cont.)

■ Execution result 2.

```
prompt> ./cpu A &  ./cpu B &  ./cpu C &  ./cpu D &  
[1] 7353  
[2] 7354  
[3] 7355  
[4] 7356  
A  
B  
D  
C  
A  
B  
D  
C  
A  
C  
B  
D  
...
```

Virtualizing Memory

- The physical memory is an array of bytes.
- A program keeps all of its data structures in memory.
 - Read memory (load):
 - Specify an address to be able to access the data
 - Write memory (store):
 - Specify the data to be written to the given address



Virtualizing Memory (Cont.)

■ A program that Accesses Memory (mem.c)

```
1      #include <unistd.h>
2      #include <stdio.h>
3      #include <stdlib.h>
4      #include "common.h"
5
6      int
7      main(int argc, char *argv[])
8      {
9          int *p = malloc(sizeof(int)); // a1: allocate some
                                         memory
10
11          assert(p != NULL);
12          printf("(%d) address of p: %08x\n",
13                getpid(), (unsigned) p); // a2: print out the
                                         address of the memory
14
15          *p = 0; // a3: put zero into the first slot of the memory
16          while (1) {
17              Spin(1);
18              *p = *p + 1;
19              printf("(%d) p: %d\n", getpid(), *p); // a4
20          }
21          return 0;
22      }
```

Virtualizing Memory (Cont.)

■ The output of the program mem.c

```
prompt> ./mem
(2134) memory address of p: 00200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

- The newly allocated memory is at address 00200000.
- It updates the value and prints out the result.

Virtualizing Memory (Cont.)

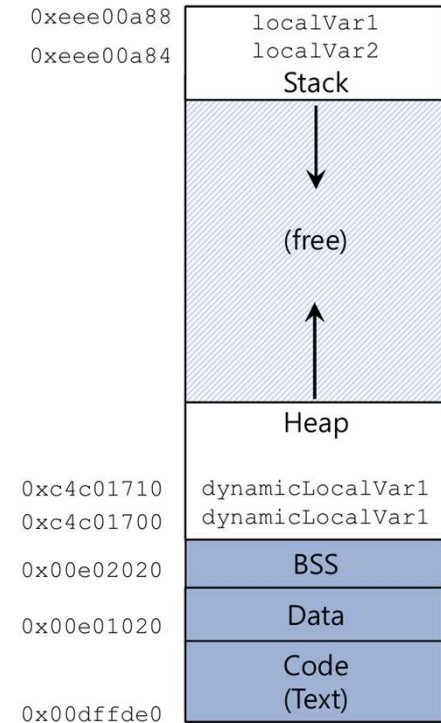
■ Running mem.c multiple times

```
prompt> ./mem & ./mem &  
[1] 24113  
[2] 24114  
(24113) memory address of p: 00200000  
(24114) memory address of p: 00200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
...
```

- It appears that each running program has its **own private memory**.
 - Each program is allocated memory at **the same address**, yet each updates the value at 0x00200000 independently.

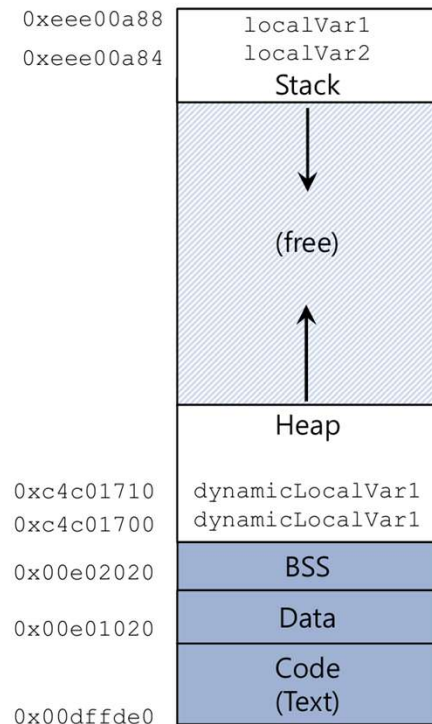
Virtualizing Memory (Cont.)

- Each process accesses its own private virtual address space.
- The OS maps **address space** onto the **physical memory**.
- Physical memory is a shared resource, managed by the OS.
- A memory reference within one running program does not affect the address space of other processes.

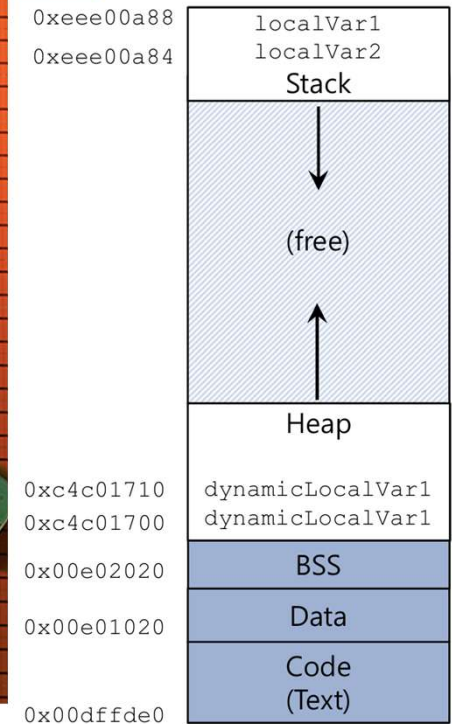
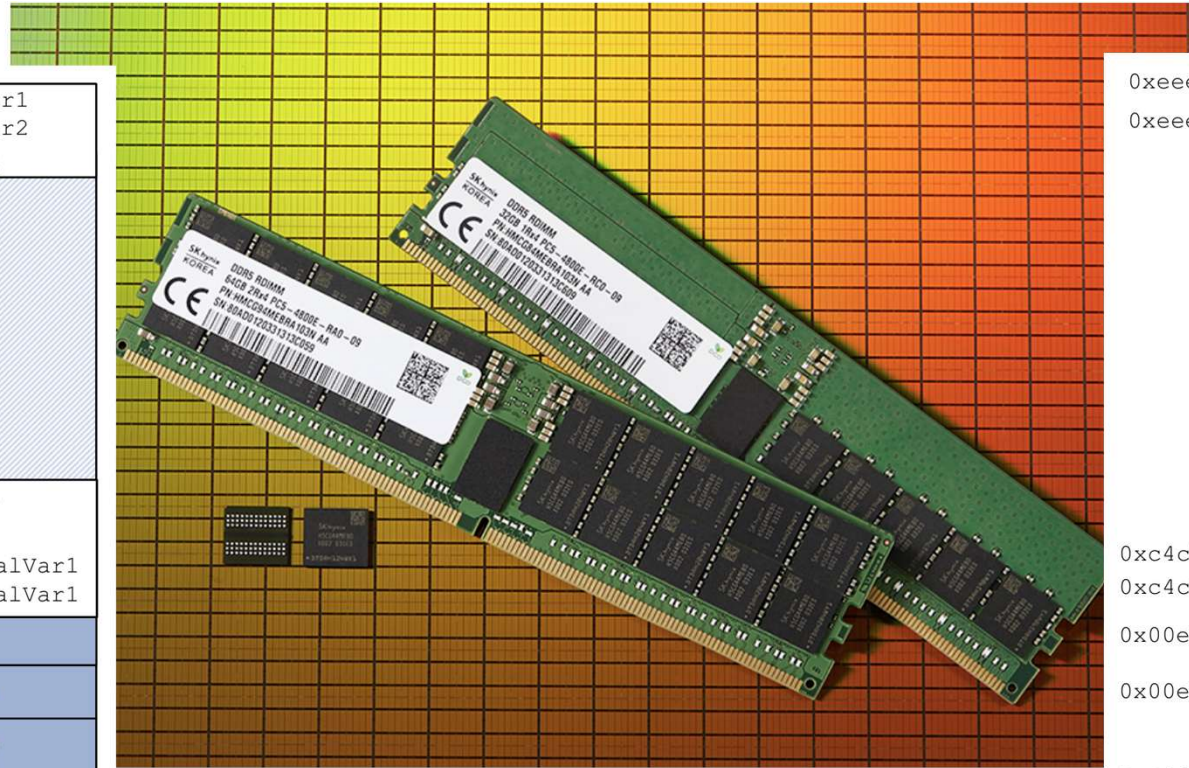


Process Virtual Address Space

Example) Virtual Memory

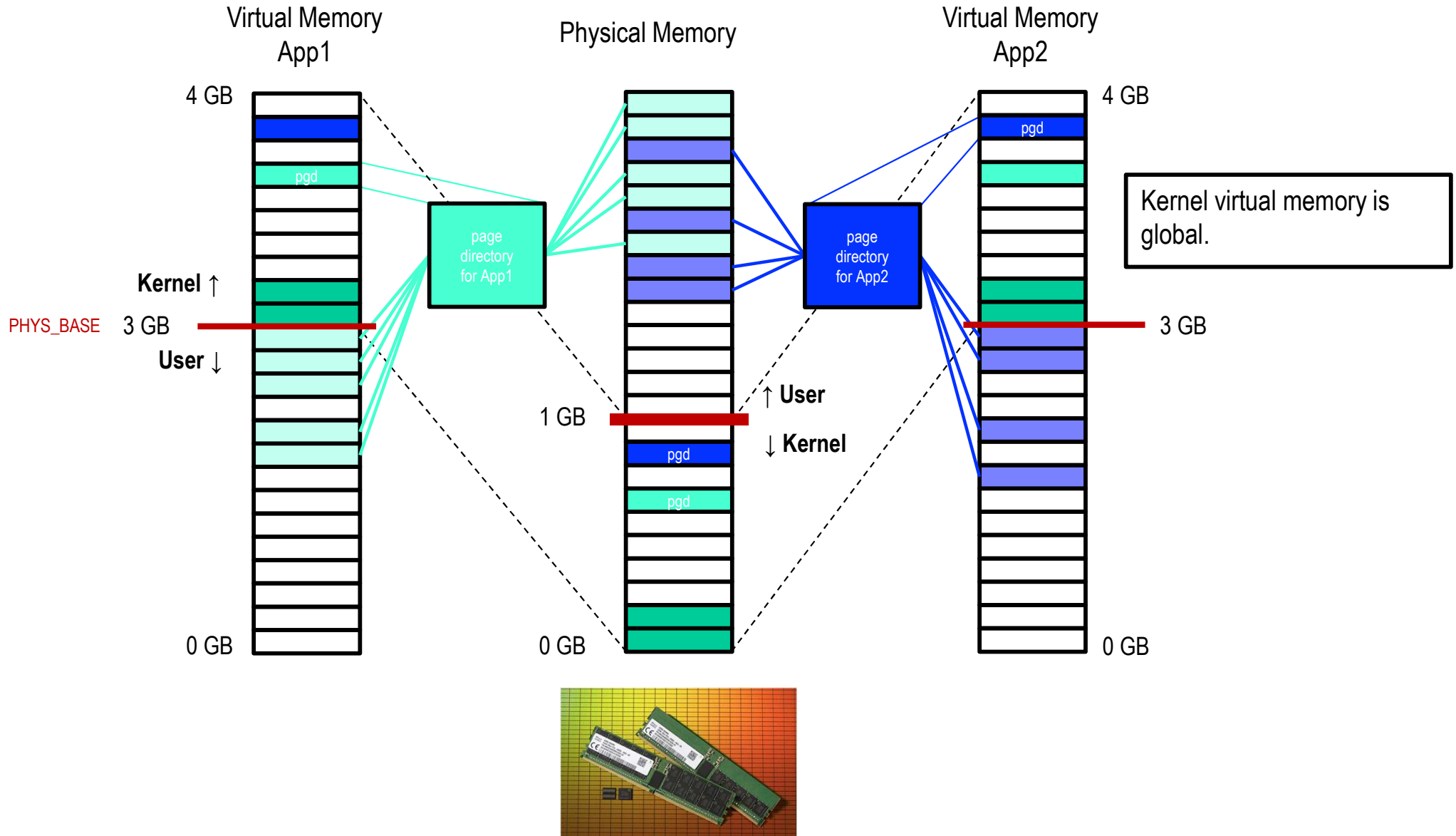


Process Virtual Address Space



Process Virtual Address Space

Example) Virtual Memory



Persistence

■ Volatile Memory

- DRAM (Main memory), SRAM (Cache)
- Stores data temporarily
- Loses all content when power is lost



■ Persistent Storage

- Preserve data even after power loss
- Requires both hardware and **software support** to achieve reliable and durable storage



■ Software: File System

- Bridges the gap between user apps and storage hardware
- Manage and organize data on disk
 - Keep track of file names, directories, and locations
 - Handle read/write requests safely
 - Ensure data consistency and recovery after crashes



Persistence (Cont.)

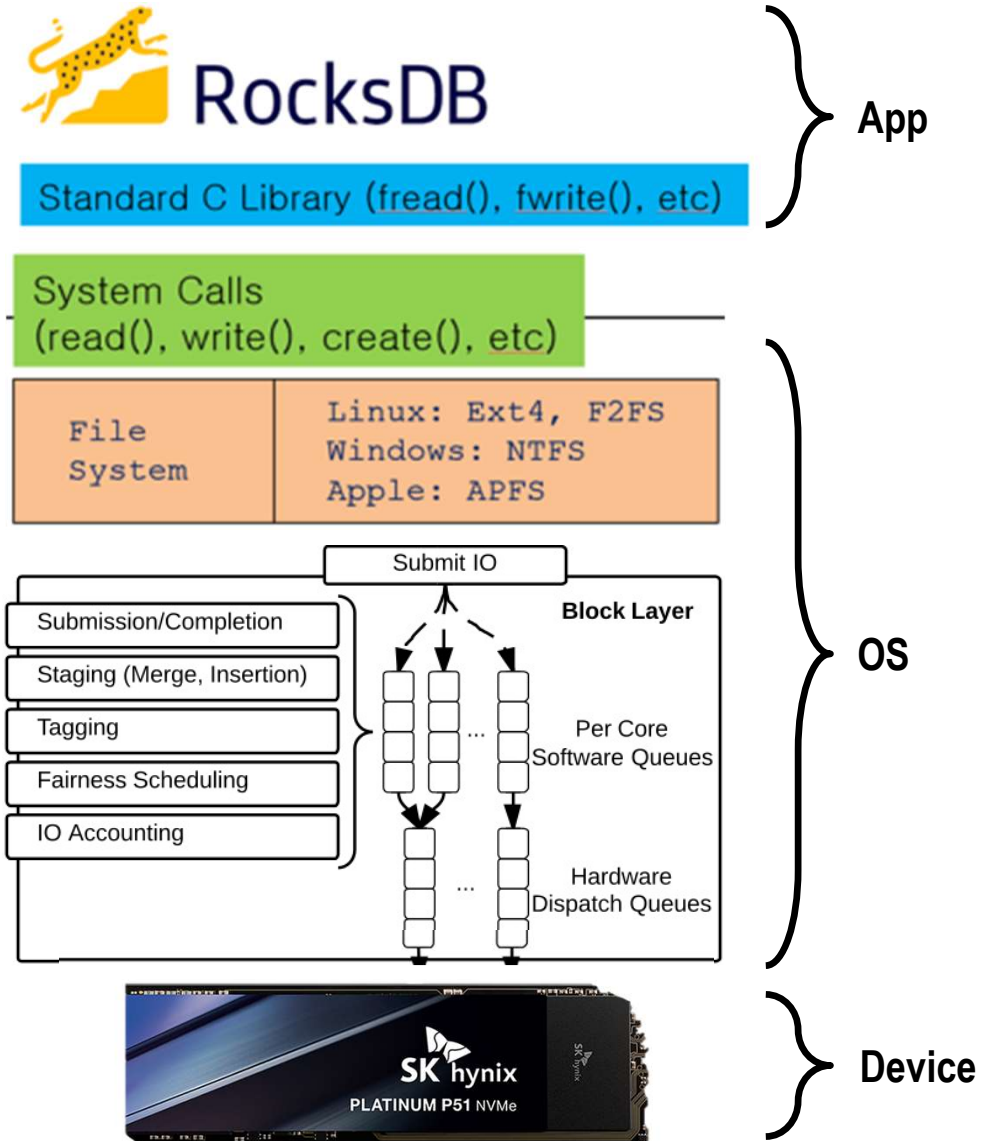
- Create a file (/tmp/file) that contains the string “hello world”

```
1      #include <stdio.h>
2      #include <unistd.h>
3      #include <assert.h>
4      #include <fcntl.h>
5      #include <sys/types.h>
6
7      int
8      main(int argc, char *argv[])
9      {
10         int fd = open("/tmp/file", O_WRONLY | O_CREAT
                        | O_TRUNC, S_IRWXU);
11         assert(fd > -1);
12         int rc = write(fd, "hello world\n", 13);
13         assert(rc == 13);
14         close(fd);
15         return 0;
16     }
```

`open()`, `write()`, and `close()` system calls are routed to the part of OS called the file system, which handles the requests

Persistence (Cont.)

- What OS does in order to write to disk?
 - Figure out **where** on disk this new data will reside
 - Issue I/O requests to the underlying storage device
- File system handles system crashes during write.
 - Journaling or copy-on-write
 - Carefully ordering writes to disk



Today

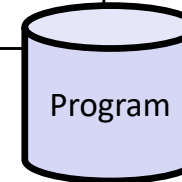
- OS Basics
- Virtualization
- **Process, Thread, and Concurrency Problem**

What is a Process ?

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <assert.h>
4 #include <fcntl.h>
5 #include <sys/types.h>
6
7 int
8 main(int argc, char *argv[])
9 {
10     int fd = open("/tmp/file",
11                  O_WRONLY | O_CREAT | O_TRUNC, S_IRWXU);
12     assert(fd > -1);
13     int rc = write(fd, "hello world\n", 13);
14     assert(rc == 13);
15     close(fd);
16     return 0;
17 }

```

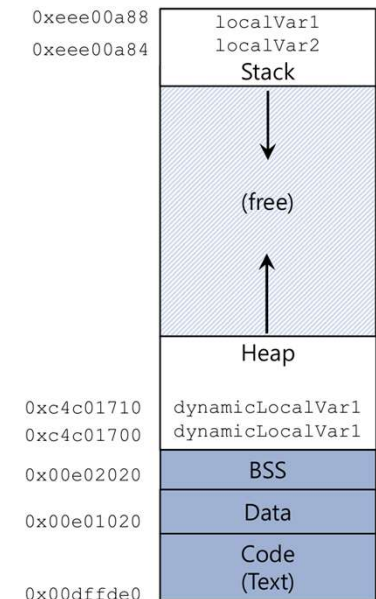


■ A process is a program in execution.

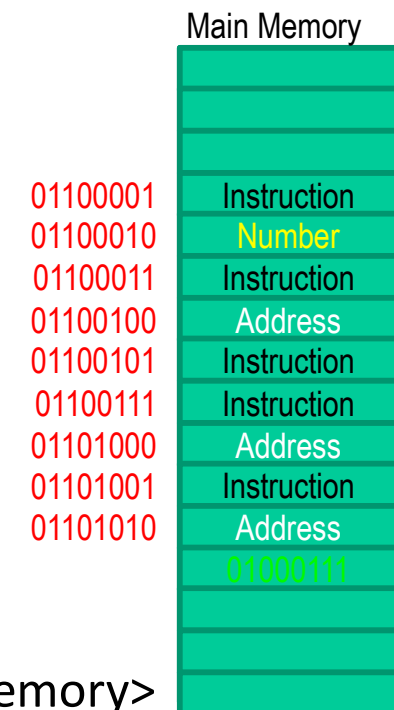
- A **program** is a passive entity — a file containing a list of instructions stored on disk (an executable file).
- A **process** is an active entity with its own program counter and runtime state.
- A program becomes a process when its executable file is loaded into memory.
 - One program can correspond to multiple processes.

■ Comprising of a process:

- Memory (address space)
 - Instructions, Data section
- Registers
 - Program counter, Stack pointer



<Process>



<Main Memory>

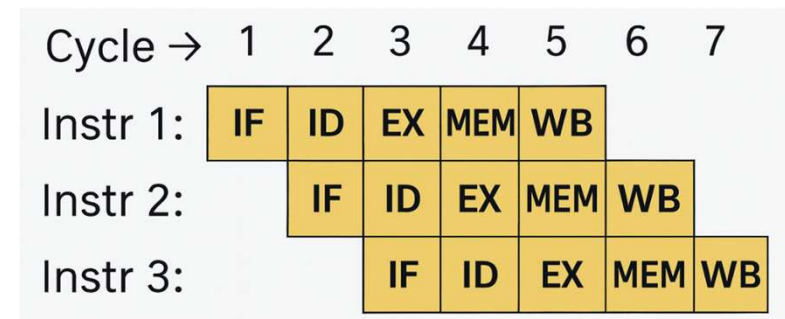
What happens when a program runs?

■ A running program executes instructions.

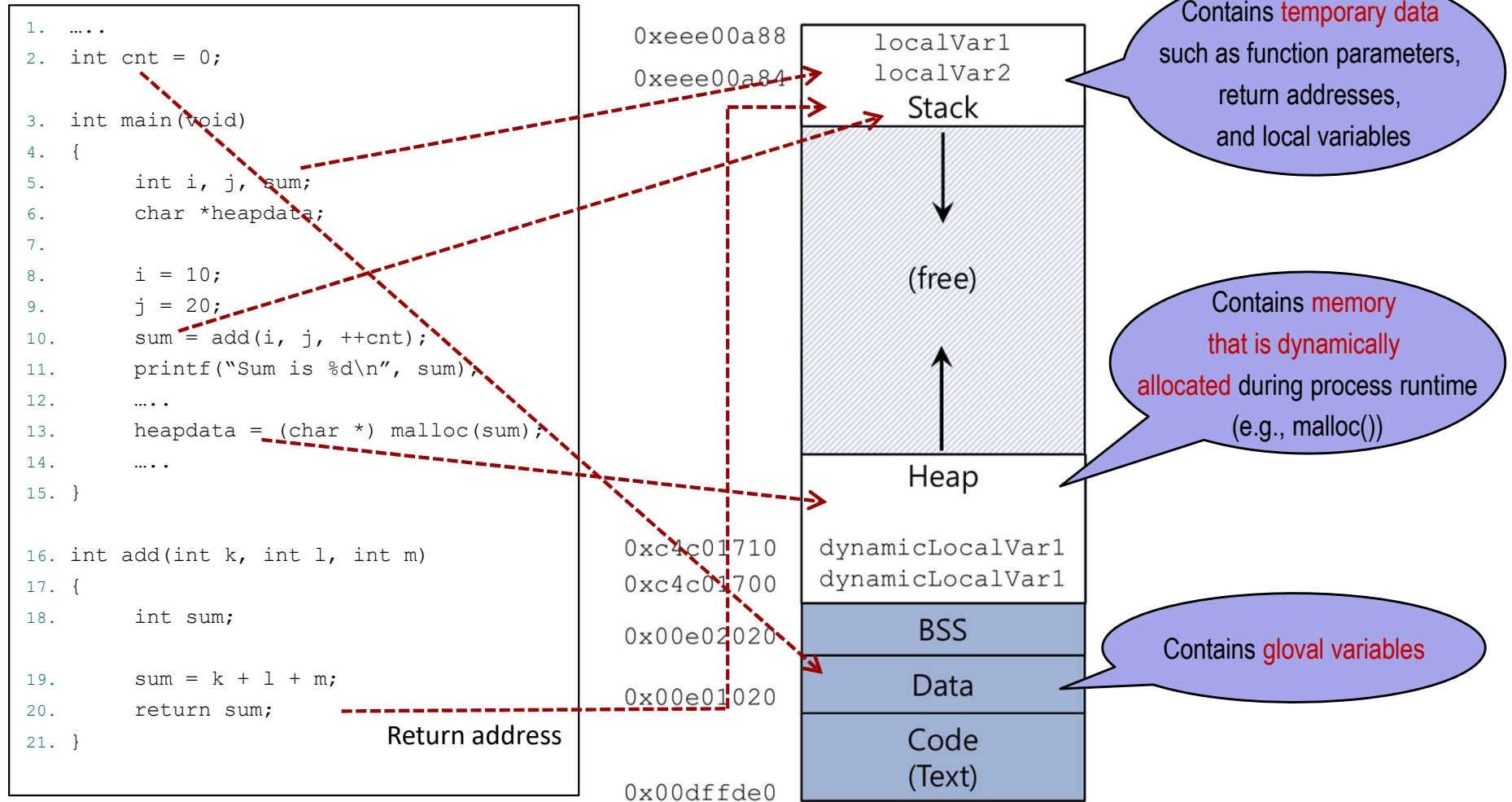
1. The processor **fetches** an instruction from memory.
2. **Decode**: Figure out which instruction this is
3. **Execute**: i.e., add two numbers, access memory, check a condition, jump to function, and so forth.
4. The processor moves on to the **next instruction** and so on.

Classic 5-Stage RISC Pipeline

- ① **IF** – Instruction Fetch: Fetch from memory
- ② **ID** – Instruction Decode / Register Fetch: Decode and read registers
- ③ **EX** – Execute: ALU operation or address calculation
- ④ **MEM** – Memory Access: Load or store data
- ⑤ **WB** – Write Back: Write result to register file



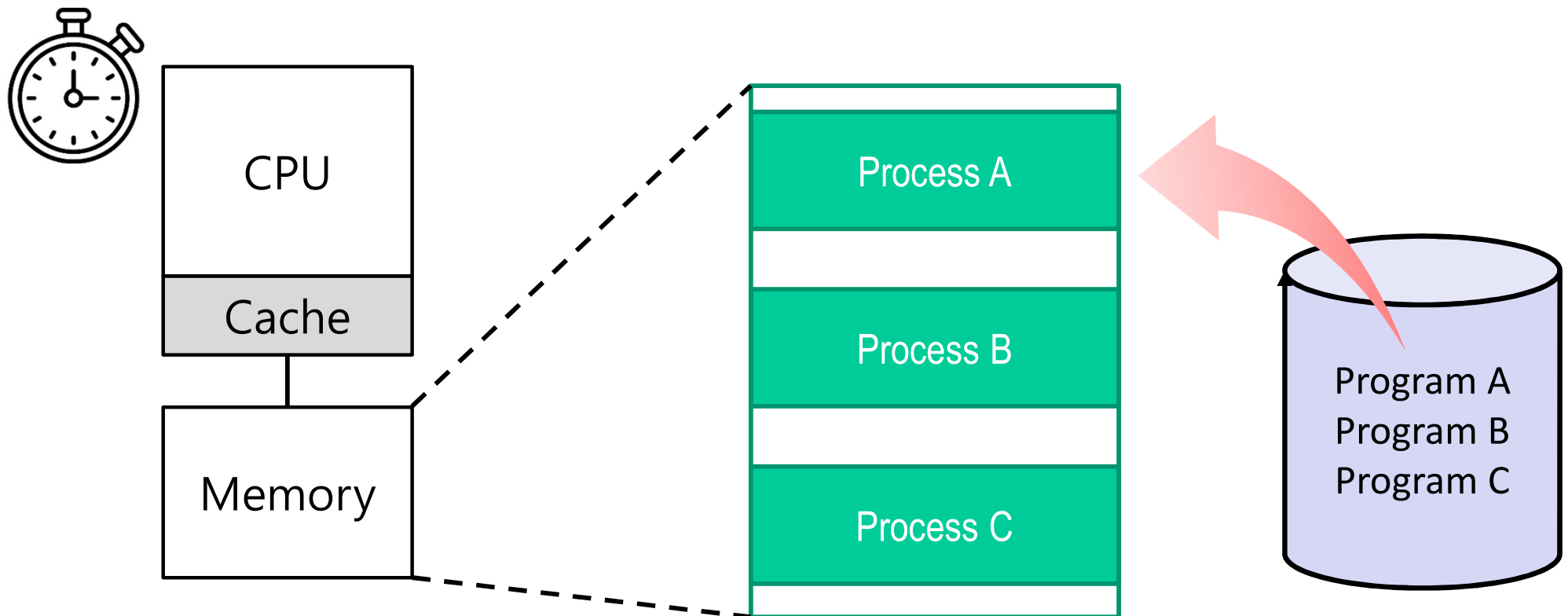
Process in Memory



How to provide the illusion of many CPUs?

■ CPU virtualizing

- The OS can promote the illusion that many virtual CPUs exist.
- **Time sharing:** Running one process, then stopping it and running another
 - The potential cost is **performance**.



Process API

- **These APIs are available on any modern OS.**
 - **Create**
 - Create a new process to run a program
 - **Destroy**
 - Halt a runaway process
 - **Wait**
 - Wait for a process to stop running
 - **Miscellaneous Control**
 - Some kind of method to suspend a process and then resume it
 - **Status**
 - Get some status info about a process

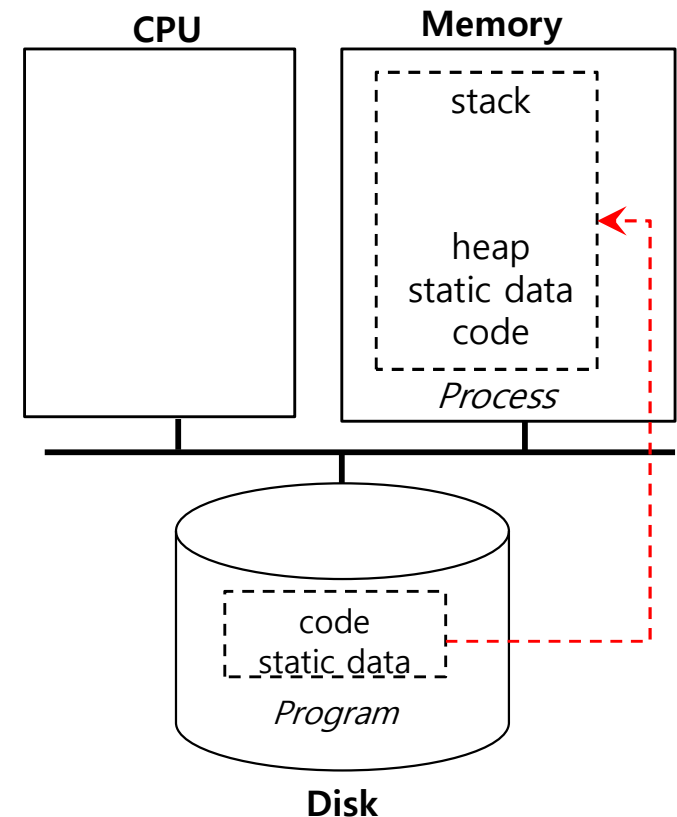
Process Creation

① Load a program code into memory, into the address space of the process.

- Programs initially reside on disk in *executable format*.
- Modern OS perform the loading process *lazily*.
 - Loading pieces of code or data only as they are needed during program execution.

② The program's run-time stack is allocated.

- Use the stack for *local variables, function parameters, and return address*.
- Initialize the stack with arguments → `argc` and the `argv` array of `main()` function



Loading:
Takes on-disk program
and reads it into the
address space of process

Process Creation (Cont.)

③ The program's heap is created.

- Used for explicitly requested dynamically allocated data.
- Program request such space by calling `malloc()` and free it by calling `free()`.

④ The OS do some other initialization tasks.

- input/output (I/O) setup
 - Each process by default has three open file descriptors.
 - Standard input, output and error

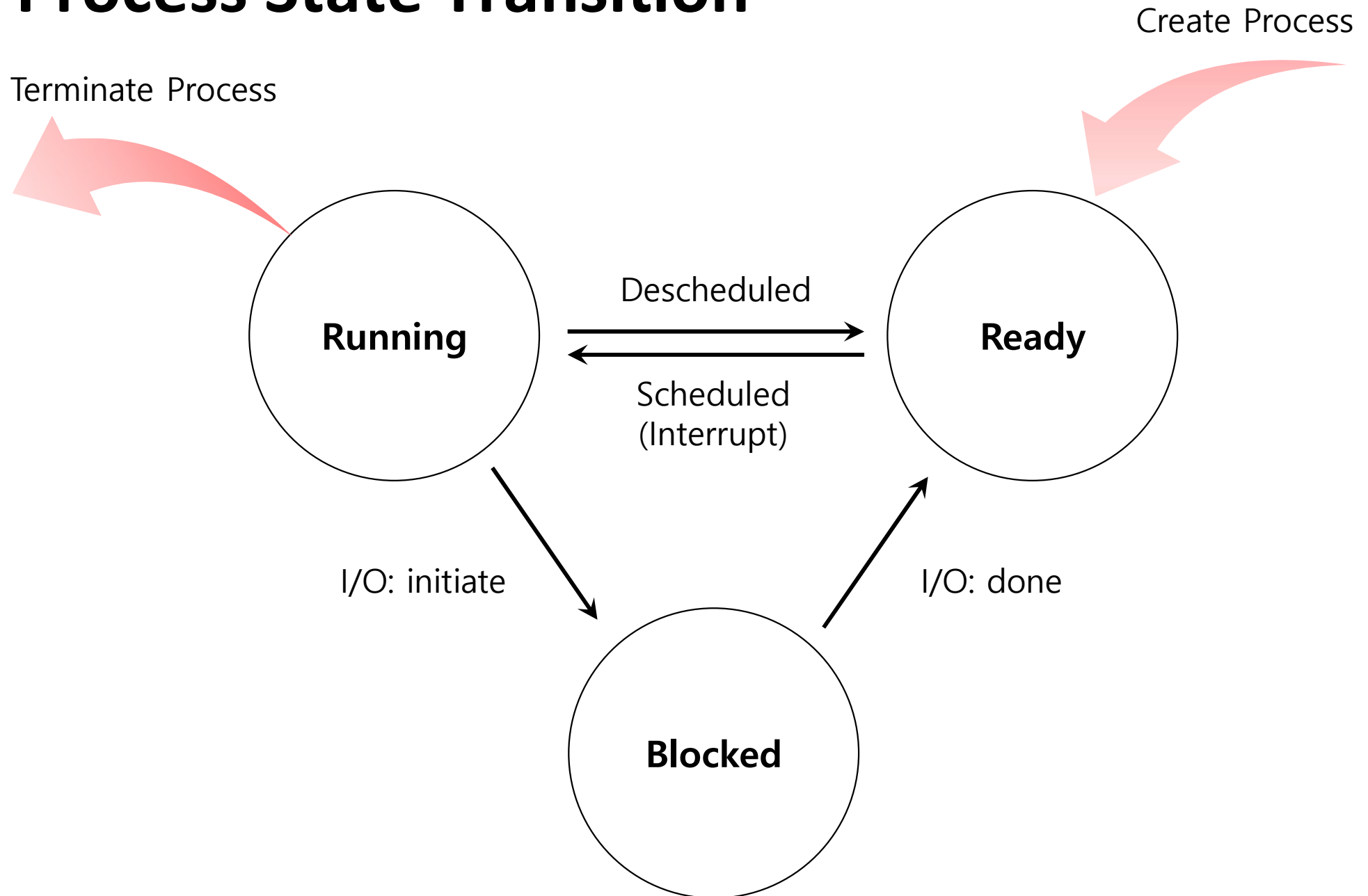
⑤ Start the program running at the entry point, namely `main()`.

- The OS *transfers control* of the CPU to the newly-created process.

Process States

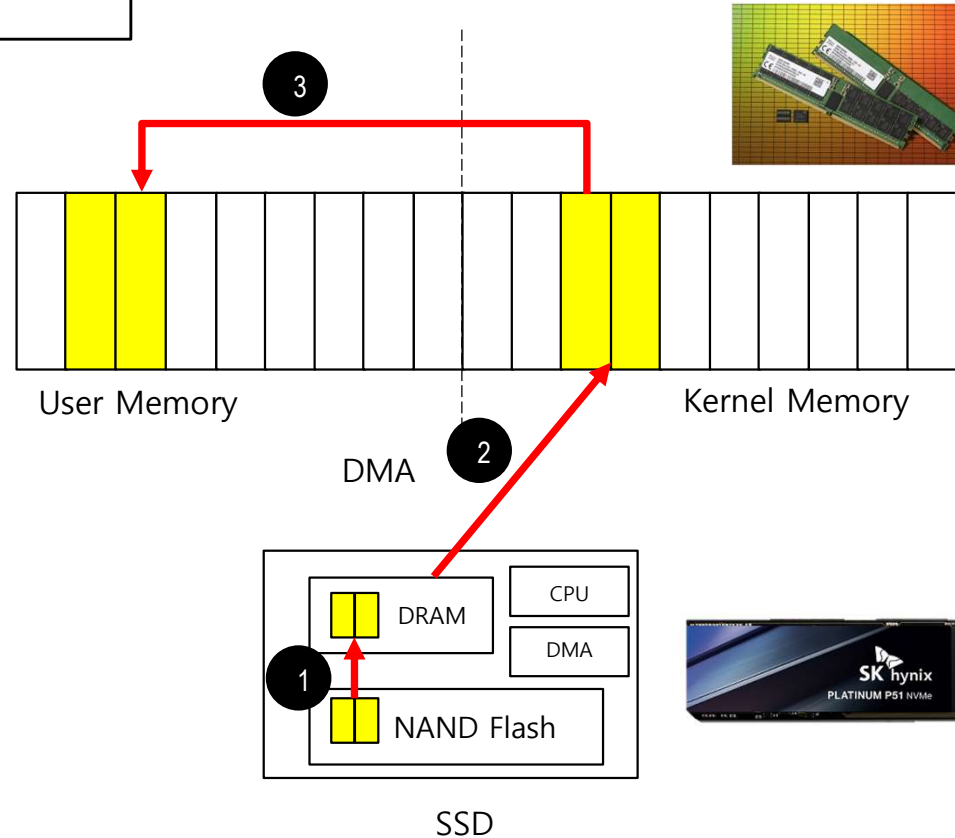
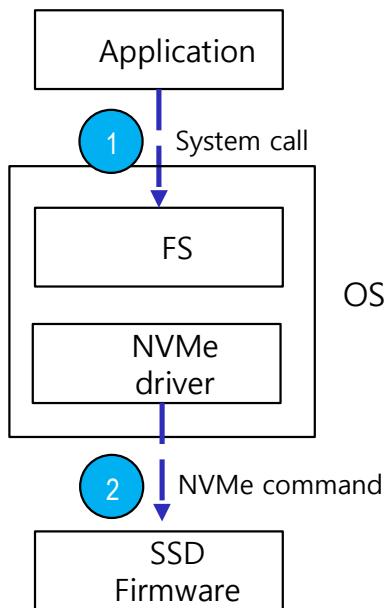
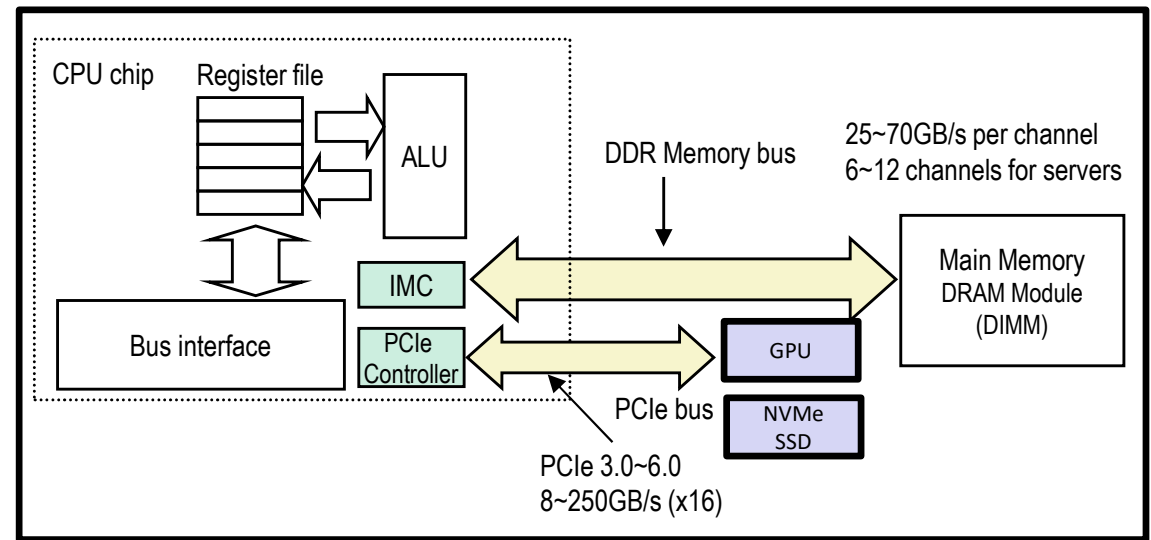
- **A process can be one of three states.**
 - **Running**
 - A process is running on a processor.
 - **Ready**
 - A process is ready to run but for some reason the OS has chosen not to run it at this given moment.
 - **Blocked**
 - A process has performed some kind of operation.
 - When a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.

Process State Transition



Example) I/O Operation Flow

```
char *buf = malloc(8192);
int fd = open("/tmp/file", O_RDONLY);
int rc = read(fd, buf, 8192);
Close(fd)
```



Example) Tracing Process State: CPU Only

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	Process ₀ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	Process ₁ now done

Figure 4.3: Tracing Process State: CPU Only

Example) Tracing Process State: CPU and I/O

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

Data structures

■ PCB(Process Control Block)

- A C-structure that contains information **about each process**.
- **Register context:** a set of registers that define the state of a process

■ Process list

- Ready processes
- Blocked processes
- Current running processes

Example) The xv6 kernel Proc Structure

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;           // Start of process memory
    uint sz;             // Size of process memory
    char *kstack;        // Bottom of kernel stack
                        // for this process

    enum proc_state state; // Process state
    int pid;              // Process ID
    struct proc *parent;  // Parent process
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;     // Current directory
    struct context context; // Switch here to run process
    struct trapframe *tf;  // Trap frame for the
                        // current interrupt
};
```

Example) Register Context in xv6

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;    // Index pointer register
    int esp;    // Stack pointer register
    int ebx;    // Called the base register
    int ecx;    // Called the counter register
    int edx;    // Called the data register
    int esi;    // Source index register
    int edi;    // Destination index register
    int ebp;    // Stack base pointer register
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
```

Example) thread structure in Pintos

```
struct thread{
    /* Owned by thread.c. */
    tid_t tid;                                /* Thread identifier. */
    enum thread_status status                 /* Thread state. */
    char name[16];                            /* Name (for debugging purposes). */
    uint8_t *stack;                          /* Saved stack pointer. */
    int priority;                            /* Priority. */
    struct list_elem allelem; /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem; /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir; /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic; /* Detects stack overflow. */
};
```


Process Control Block in Linux

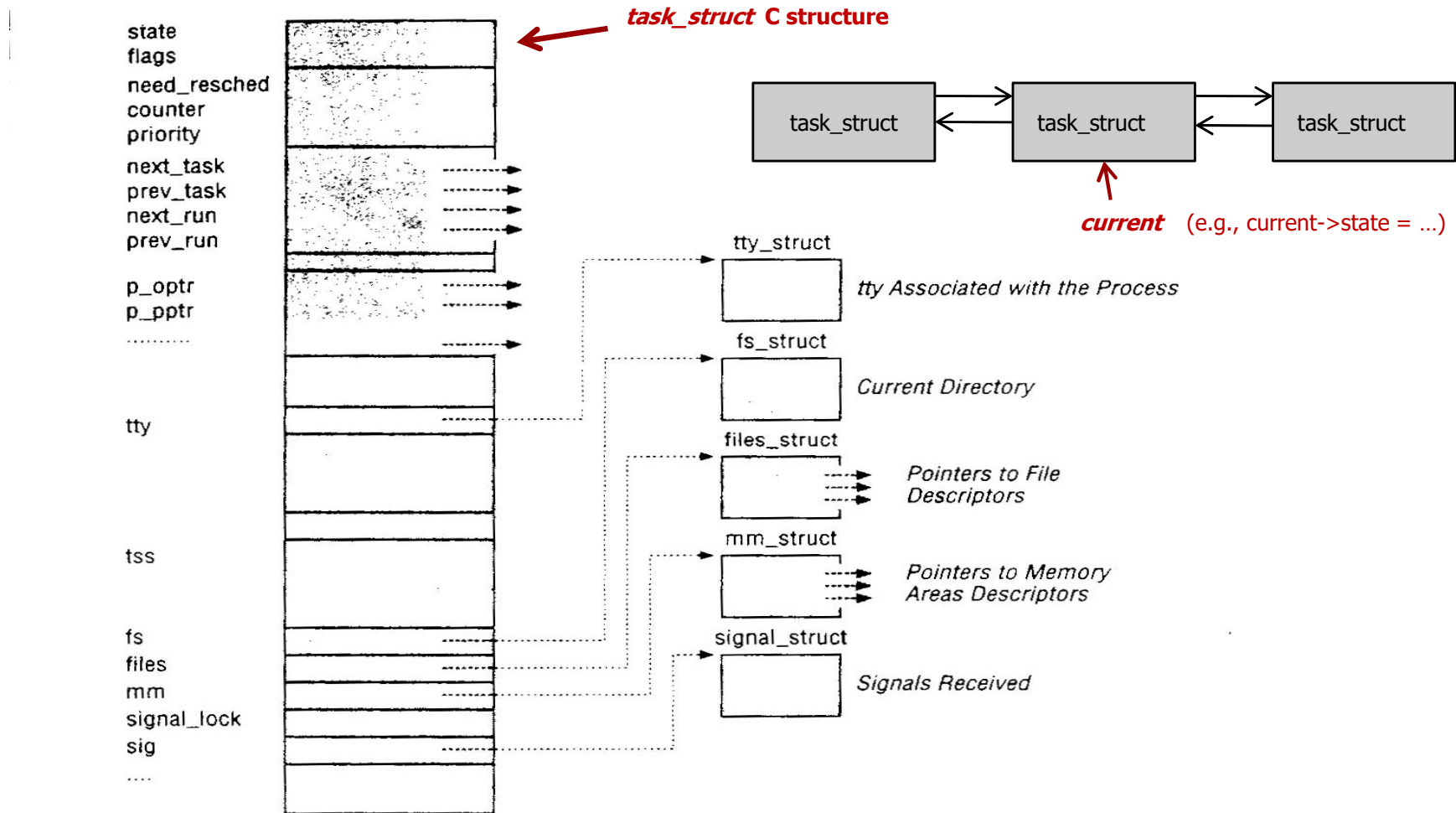
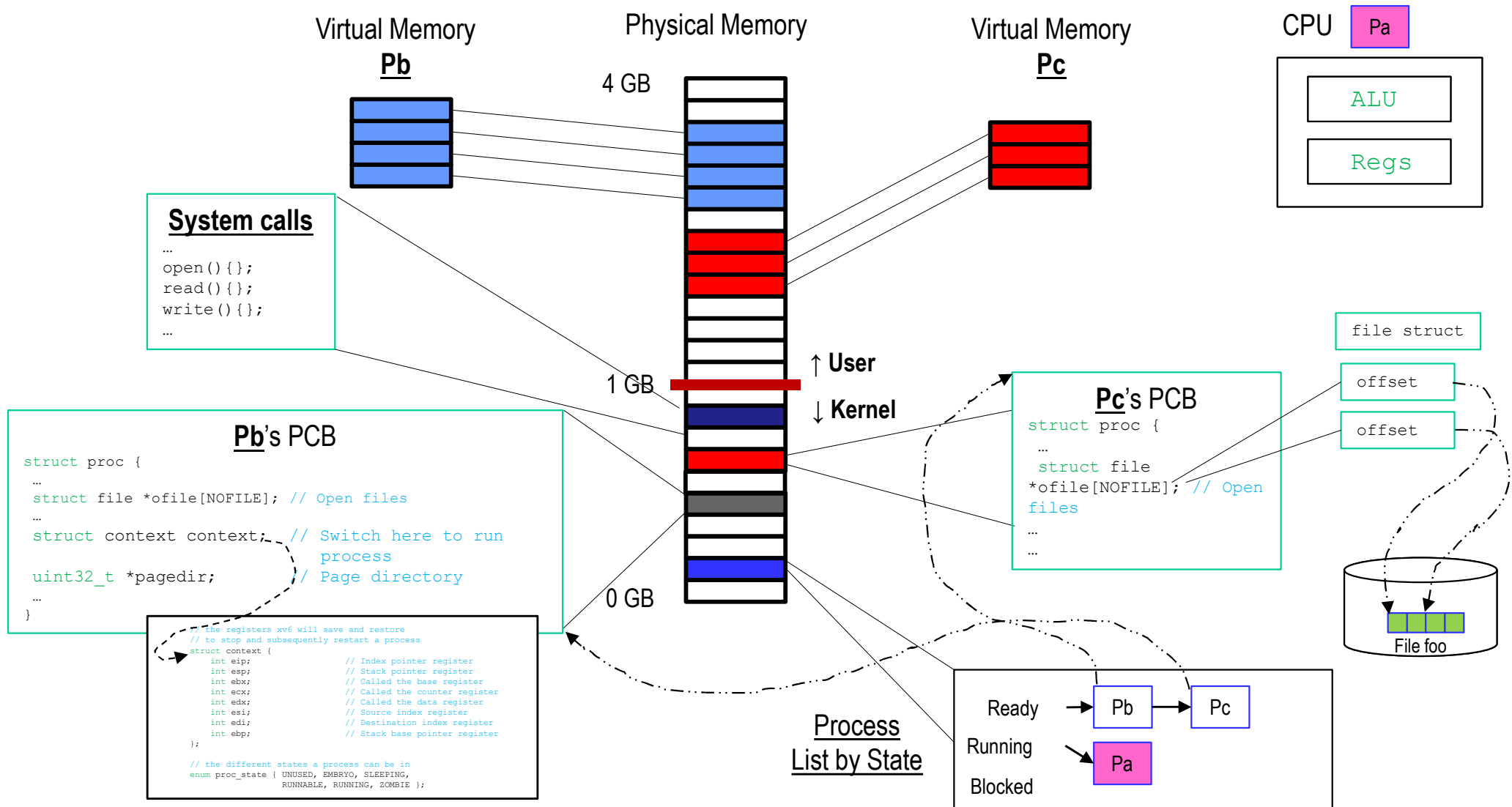


Figure 3-1. The Linux process descriptor

가정:

- 물리적인 메모리 (Physical Memory) 크기 4GB
- 페이지 크기: 4KB (아래 그림에서 한 개 박스!)
- **가상메모리 (응용이 사용 가능한) 최대 16KB (그 이상 허용 불가!)**
- 가상메모리 -> 물리적인 메모리 매핑: **linear mapping** (1-1 매핑)
 - ✓ Directory 같은 구조는 필요 없음. 왜냐하면, linear mapping이므로 첫 페이지가 mapping되어 있는 물리적인 페이지 주소만 알고 있으면 됨
- 아래 그림에서 프로세스 가상 메모리 커널 부분은 설명의 용이를 위해 hide했음



What is a Thread? — Flow of Instructions

■ Concept

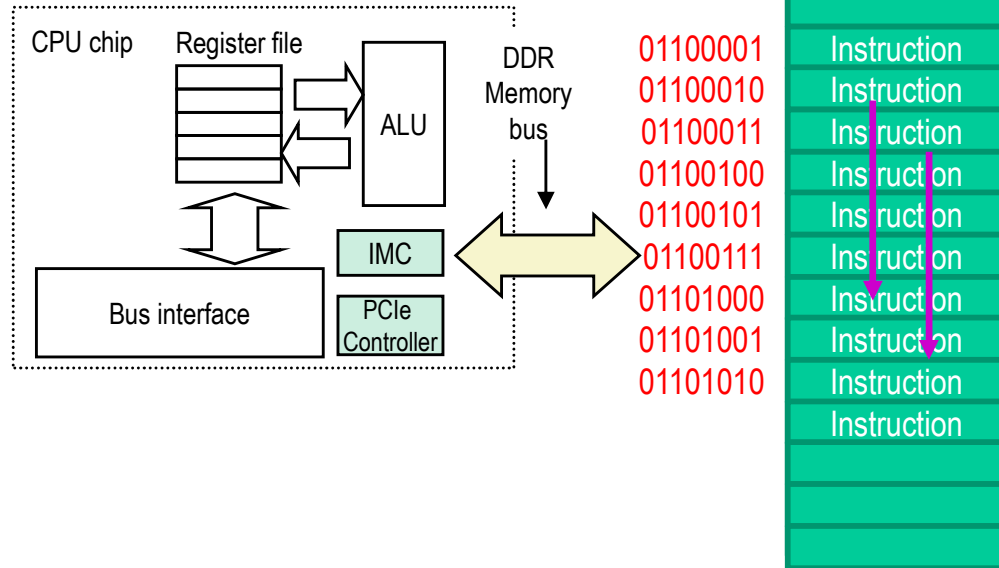
- A thread represents a *sequence of instructions* executed by the CPU.
- It defines how a program's instructions flow through time — the path of execution.
- Each program may have one or more threads, each following its own control flow.

■ Structure

- Program: A static set of instructions (code).
- Thread: The ***active execution flow*** of those instructions.
- Each thread has:
 - **Program Counter (PC):** tracks the next instruction to execute
 - **Stack:** stores function calls and local variables
 - **Registers:** hold temporary computation data

Example

Two Threads
=
Two Instruction
Flows



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "common.h"
4
5 volatile int counter = 0;
6 int loops;
7
8 void *worker(void *arg) {
9     int i;
10     for (i = 0; i < loops; i++) {
11         counter++;
12     }
13     return NULL;
14 }
15
16 int
17 main(int argc, char *argv[])
18 {
19     if (argc != 2) {
20         fprintf(stderr, "usage: threads <value>\n");
21         exit(1);
22     }
23     loops = atoi(argv[1]);
24     pthread_t p1, p2;
25     printf("Initial value : %d\n", counter);
26
27     Pthread_create(&p1, NULL, worker, NULL);
28     Pthread_create(&p2, NULL, worker, NULL);
29     Pthread_join(p1, NULL);
30     Pthread_join(p2, NULL);
31     printf("Final value : %d\n", counter);
32     return 0;
33 }
```

- A single thread executes these instructions in order, one after another.
- **Multiple threads** can run **different instruction flows** within the **same program**.

Concurrency Problem

- The OS handles many processes simultaneously by rapidly switching between them.
- This time-sharing creates the illusion that programs run at the same time.
- In **multi-threaded programs**, multiple threads within one process run concurrently.
- Such parallel execution can cause concurrency issues like **race conditions**.

Concurrency Example

■ A Multi-threaded Program (thread.c)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include "common.h"
4
5      volatile int counter = 0;
6      int loops;
7
8      void *worker(void *arg) {
9          int i;
10         for (i = 0; i < loops; i++) {
11             counter++;
12         }
13         return NULL;
14     }
15
16     int
17     main(int argc, char *argv[])
18     {
19         if (argc != 2) {
20             fprintf(stderr, "usage: threads <value>\n");
21             exit(1);
22         }
```

Concurrency Example (Cont.)

```
23         loops = atoi(argv[1]);
24         pthread_t p1, p2;
25         printf("Initial value : %d\n", counter);
26
27         Pthread_create(&p1, NULL, worker, NULL);
28         Pthread_create(&p2, NULL, worker, NULL);
29         Pthread_join(p1, NULL);
30         Pthread_join(p2, NULL);
31         printf("Final value : %d\n", counter);
32         return 0;
33     }
```

- The main program creates **two threads**.
 - Thread: a function running within the same memory space.
 - Each thread start running in a routine called `worker()`.
 - `worker()`: increments a counter

Declaring a variable with the **volatile** keyword

■ **volatile int counter = 0;**

- In C, declaring a variable as volatile tells the compiler not to optimize away memory accesses for that variable.

■ When can values change unexpectedly?

- Interrupt Service Routines (ISRs) that update variables asynchronously
- Shared variables in multi-threaded or multi-core programs

```
volatile int flag = 0;

void interrupt_handler() {
    flag = 1; // Changed inside ISR
}

int main() {
    while (flag == 0) {
        // Because of volatile, flag is
always
        read from memory
    } // Loop exits once flag becomes 1
}
```

Without volatile, the compiler might optimize the loop into an infinite loop, assuming flag never changes.

Example

C

```
int x;  
while (x == 0) { }
```

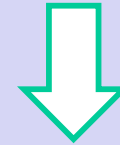


asm

```
mov eax, DWORD PTR [x]  
.L1:  
cmp eax, 0  
je .L1
```

C

```
volatile int x;  
while (x == 0) { }
```



asm

```
.L1:  
mov eax, DWORD PTR [x]  
    ; always reload from memory  
cmp eax, 0  
je .L1
```

Concurrency Example (Cont.)

- **loops** determines how many times each of the two workers will increment the shared counter in a loop.

- `loops: 1000.`

```
prompt> gcc -o thread thread.c -Wall -pthread
prompt> ./thread 1000
Initial value : 0
Final value : 2000
```

- `loops: 100000.`

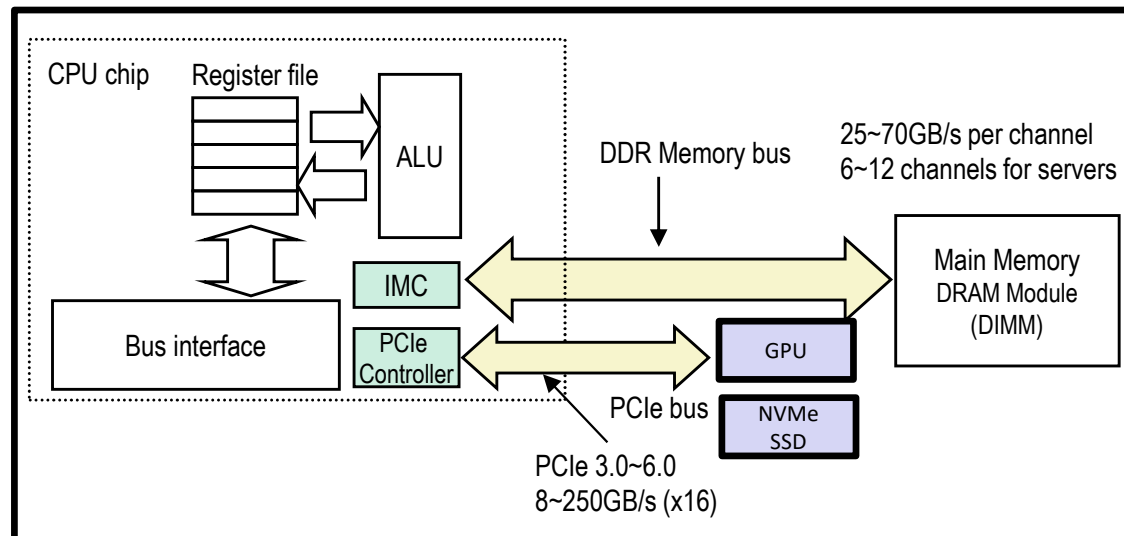
```
prompt> ./thread 100000
Initial value : 0
Final value : 143012 // huh??
prompt> ./thread 100000
Initial value : 0
Final value : 137298 // what the??
```

Why is this happening?

- **Increment a shared counter → take three instructions.**

- ① Load the value of the counter from memory into register.
- ② Increment it
- ③ Store it back into memory

- **These three instructions do not execute *atomically*.**
→ **Concurrency problem happens.**



Assembly Code for Counter Loop

C code for counter loop in thread i

```
for (i = 0; i < loops; i++)
    counter++;
```

Asm code for thread i

H_i : Head

L_i : Load cnt

U_i : Update cnt

S_i : Store cnt

T_i : Tail

```
movq    (%rdi), %rcx
testq   %rcx, %rcx
jle     .L2
movl    $0, %rax
```

.L3:

```
movq    cnt(%rip), %rdx
addq    $1, %rdx
movq    %rdx, cnt(%rip)
```

```
addq    $1, %rax
cmpq    %rcx, %rax
jne     .L3
```

.L2:

- # 1. %rdi가 가리키는 메모리 주소에서 8바이트 값을 읽어 %rcx에 저장
- # 2. %rcx == 0 인지 확인 (ZF 설정)
- # 3. %rcx <= 0이면 .L2로 점프 (<= 비교는 signed 기준)
- # 4. 그렇지 않으면 %rax를 0으로 초기화

.L3:

- # 5. 전역 변수 cnt 값을 %rdx에 로드 (RIP-relative addressing)
- # 6. %rdx에 1을 더함
- # 7. 다시 cnt에 %rdx를 저장 (cnt = cnt + 1)

- # 8. 루프 인덱스 %rax += 1
- # 9. %rax와 %rcx를 비교
- # 10. 같지 않으면 다시 .L3으로 루프

- # 11. 루프 종료 지점

Concurrent Execution

- **Key idea:** In general, any sequentially consistent interleaving is possible, but some give an unexpected result!

- I_i denotes that thread i executes instruction I
- $\%rdx_i$ is the content of $\%rdx$ in thread i 's context

i (thread)	$instr_i$	$\%rdx_1$	$\%rdx_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
1	S_1	1	-	1
2	H_2	-	-	1
2	L_2	-	1	1
2	U_2	-	2	1
2	S_2	-	2	2
2	T_2	-	2	2
1	T_1	1	-	2



Thread 1
critical section



Thread 2
critical section

OK

Concurrent Execution (cont)

- **Incorrect ordering: two threads increment the counter, but the result is 1 instead of 2**

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁	-	-	0
1	L ₁	0	-	0
1	U ₁	1	-	0
2	H ₂	-	-	0
2	L ₂	-	0	0
1	S ₁	1	-	1
1	T ₁	1	-	1
2	U ₂	-	1	1
2	S ₂	-	1	1
2	T ₂	-	1	1

 Thread 1
critical section

 Thread 2
critical section

Oops!

Concurrent Execution (cont)

■ How about this ordering?

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁			0
1	L ₁	0		
2	H ₂			
2	L ₂		0	
2	U ₂		1	
2	S ₂		1	1
1	U ₁	1		
1	S ₁	1		1
1	T ₁			1
2	T ₂			1

Oops!

OS Basics and Process: Summary

■ OS = Virtualization + Resource Management

- Transforms hardware (CPU, memory, disk) into virtual and manageable resources
- Makes programs run efficiently, safely, and concurrently

■ Process vs. Thread

- Process: A program in execution (code/data, heap, stack, registers)
 - States: Running, Ready, Blocked
- Thread: A *sequence of instructions* executed by the CPU
- Threads within the same process share its code, data, heap, and file resources.

■ Concurrency Problem

- Multiple threads can cause race conditions (counter++)